

Отчёт по лабораторной работе 4

Соловьев Богдан Михайлович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Параметры модели	7
4	Выполнение лабораторной работы	9
5	Модель SIR	11
5.1	Параметры эксперимента	11
5.2	Инициализация модели	12
5.3	Запуск симуляции	13
5.4	Сохранение результатов	14
5.5	Визуализация	16
6	Сканирование параметра β в модели SIR	18
6.1	Функция для запуска одного эксперимента	18
6.2	Параметры эксперимента	20
6.3	Генерация списка параметров	21
6.4	Запуск экспериментов	22
6.5	Сохранение результатов	23
6.6	Усреднение по повторам	23
6.7	Визуализация результатов	24
7	Оптимизация параметров модели SIR	33
7.1	Целевая функция	33
7.2	Запуск оптимизации	35
7.3	Результаты оптимизации	36
7.4	дополнительное задание	38
8	Выводы	43
	Список литературы	44

Список иллюстраций

4.1	sir_model	9
4.2	Результат	10
5.1	выполнение	17
6.1	migration	26
6.2	parameters	32
7.1	parameters	38
7.2		38
7.3	график	39
7.4		39
7.5		40
7.6		41
7.7		41
7.8		42
7.9	ВЫВОД КОДА	42

Список таблиц

```
using Pkg
Pkg.activate("C:/Users/bogda/work/study/2026-1/2026-1--study--simul
using Agents, DataFrames, Plots, StatsBase
println("XXXXXX XXXXXXXXXXXX: ", Base.active_project())
```

```
XXXXXX XXXXXXXXXXXX: C:\Users\bogda\work\study\2026-1\2026-
1--study--simulationmodeling\labs\lab04\project\Project.toml
```

```
Activating project at `C:\Users\bogda\work\study\2026-
1\2026-1--study--simulationmodeling\labs\lab04\project`
```

1 Цель работы

Создадим агентную модель распространения инфекционного заболевания на основе классической компартментальной модели SIR (Susceptible-Infectious-Recovered).

Модель будет реализована с использованием пакета Agents.jl.

В отличие от классической модели на дифференциальных уравнениях, агентный подход позволит учесть индивидуальные характеристики, пространственную структуру и стохастичность процессов

2 Задание

Создать рабочий каталог для кода.

Установить необходимые пакеты.

Выполнить предложенный код.

Преобразовать код в литературный стиль.

Сгенерировать из литературного кода:

чистый код;

jupyter notebook;

документацию в формате Quarto.

Выполнить код из jupyter notebook.

Интегрировать документацию в формате Quarto в отчёт.

Добавить в код в литературном стиле вычисление для набора параметров.

Сгенерировать из литературного кода с параметрами:

чистый код;

jupyter notebook;

документацию в формате Quarto.

Выполнить код из jupyter notebook с параметрами.

Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

Модель SIR, предложенная Кермаком и Маккендриком в 1927 году, описывает динамику эпидемии в популяции, разделённой на три группы:

(Susceptible) — восприимчивые к заболеванию индивиды;

(Infectious) — инфицированные, способные заражать восприимчивых;

(Recovered) — выздоровевшие (или умершие), получившие иммунитет и более не участвующие в распространении.

4.1.1.2 Ограничения классического подхода

Несмотря на широкое применение, модель на ОДУ имеет ряд ограничений:

Однородность популяции — все индивиды считаются одинаковыми.

Отсутствие пространственной структуры — предполагается полное перемешивание.

Детерминированность — не учитываются случайные флуктуации.

Непрерывность — количество людей рассматривается как непрерывная величина.

4.1.1.3 Преимущества агентного подхода

Агентное моделирование позволяет преодолеть эти ограничения:

Каждый индивид моделируется отдельно с уникальными характеристиками.

Взаимодействия происходят локально в пространстве или социальной сети.

Процессы носят стохастический характер.

Можно учитывать гетерогенность контактов, мобильность, меры контроля.

3.1 Параметры модели

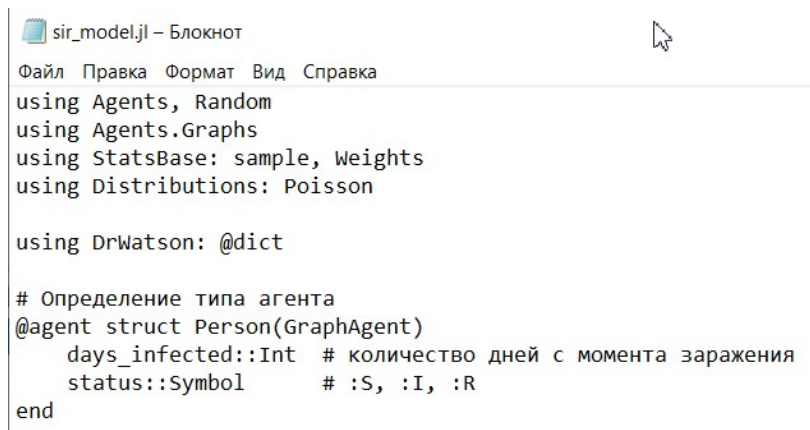
N_s — вектор численности населения по локациям;

β_{und} — вероятность передачи для невыявленных случаев;
 β_{det} — вероятность передачи для выявленных случаев;
infection_period — длительность заболевания (дней);
detection_time — время до выявления заболевания;
death_rate — вероятность летального исхода;
reinfection_probability — вероятность повторного заражения;
migration_rates — матрица вероятностей миграции между локациями.

4 Выполнение лабораторной работы

Создаю пространство для выполнения лабораторной. Для этого создаю `setup_report`, потом `add_packages` и `tangle`. (Ничего нового)

Создаю файл `sir_model` в папке `src` (рис. 4.1).



```
sir_model.jl – Блокнот
Файл Правка Формат Вид Справка
using Agents, Random
using Agents.Graphs
using StatsBase: sample, weights
using Distributions: Poisson

using DrWatson: @dict

# Определение типа агента
@agent struct Person(GraphAgent)
    days_infected::Int # количество дней с момента заражения
    status::Symbol      # :S, :I, :R
end
```

Рисунок 4.1: `sir_model`

Создаю скрипт `sir_run_basic` и запускаю его. Пик эпидемии где-то на 18 день(рис. 4.2).

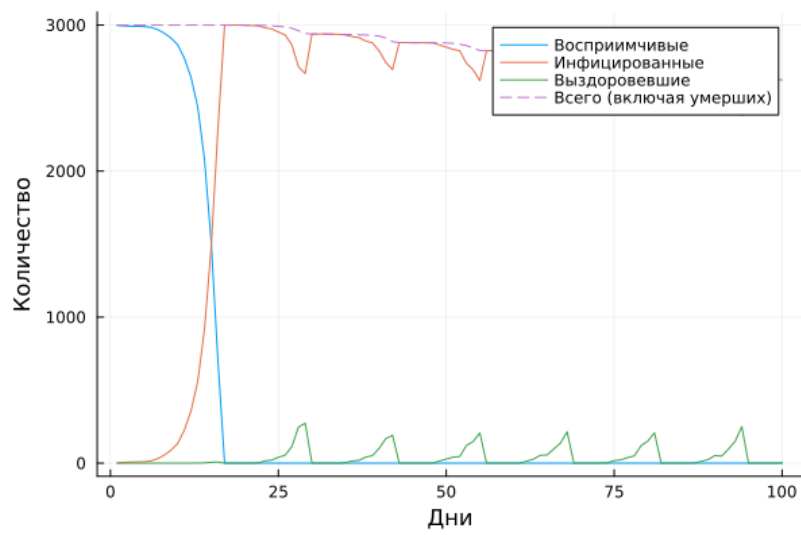


Рисунок 4.2: Результат

5 Модель SIR

Загружаем проект и необходимые пакеты.

```
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots
using JLD2
```

```
└ Warning: DrWatson could not find find a project file by recursively cl
⊠ (given dir: C:\Users\bogda)
└ @ DrWatson C:\Users\bogda\.julia\packages\DrWatson\2QF5p\src\proje
```

Подключаем файл с определением модели.

```
include(scrdir("sir_model.jl"))
```

```
total_count (generic function with 1 method)
```

5.1 Параметры эксперимента

Задаём параметры для симуляции.

```
params = Dict(
    :Ns => [1000, 1000, 1000],
```

```
:x_und => [0.5, 0.5, 0.5],
:x_det => [0.05, 0.05, 0.05],
:infection_period => 14,
:detection_time => 7,
:death_rate => 0.02,
:reinfection_probability => 0.1,
:Is => [0, 0, 1],
:seed => 42,
:n_steps => 100,
)
```

Dict{Symbol, Any} with 10 entries:

```
:Is           => [0, 0, 1]
:seed         => 42
:infection_period => 14
:n_steps      => 100
:x_und        => [0.5, 0.5, 0.5]
:Ns           => [1000, 1000, 1000]
:detection_time => 7
:reinfection_probability => 0.1
:x_det        => [0.05, 0.05, 0.05]
:death_rate   => 0.02
```

5.2 Инициализация модели

Создаём начальную модель с заданными параметрами.

```
model = initialize_sir(; params...)
```

StandardABM with 3000 agents of type Person

agents container: Dict

space: GraphSpace with 3 positions and 3 edges

scheduler: fastest

properties: C, infection_period, ∅_und, Ns, migration_rates, detection

5.3 Запуск симуляции

Подготавливаем массивы для хранения данных.

```
times = Int[]  
S_vals = Int[]  
I_vals = Int[]  
R_vals = Int[]  
total_vals = Int[]
```

Int64[]

Запускаем симуляцию вручную на указанное количество шагов.

```
for step = 1:params[:n_steps]  
    Agents.step!(model, 1)  
  
    push!(times, step)  
    push!(S_vals, susceptible_count(model))  
    push!(I_vals, infected_count(model))  
    push!(R_vals, recovered_count(model))  
    push!(total_vals, total_count(model))  
end
```

5.4 Сохранение результатов

Создаём DataFrame для удобного хранения.

```
agent_df = DataFrame(time = times, susceptible = S_vals, infected =  
model_df = DataFrame(time = times, total = total_vals)
```

	time	total
	Int64	Int64
1	1	3000
2	2	3000
3	3	3000
4	4	3000
5	5	3000
6	6	3000
7	7	3000
8	8	3000
9	9	3000
10	10	3000
11	11	3000
12	12	3000
13	13	3000
14	14	3000
15	15	3000
16	16	3000
17	17	3000
18	18	3000
19	19	3000
20	20	2999
21	21	2998
22	22	2998
23	23	2997
24	24	2995
25	25	2992
26	26	2987
27	27	2979
28	28	2961
29	29	2942
30	30	2939
...	...	...

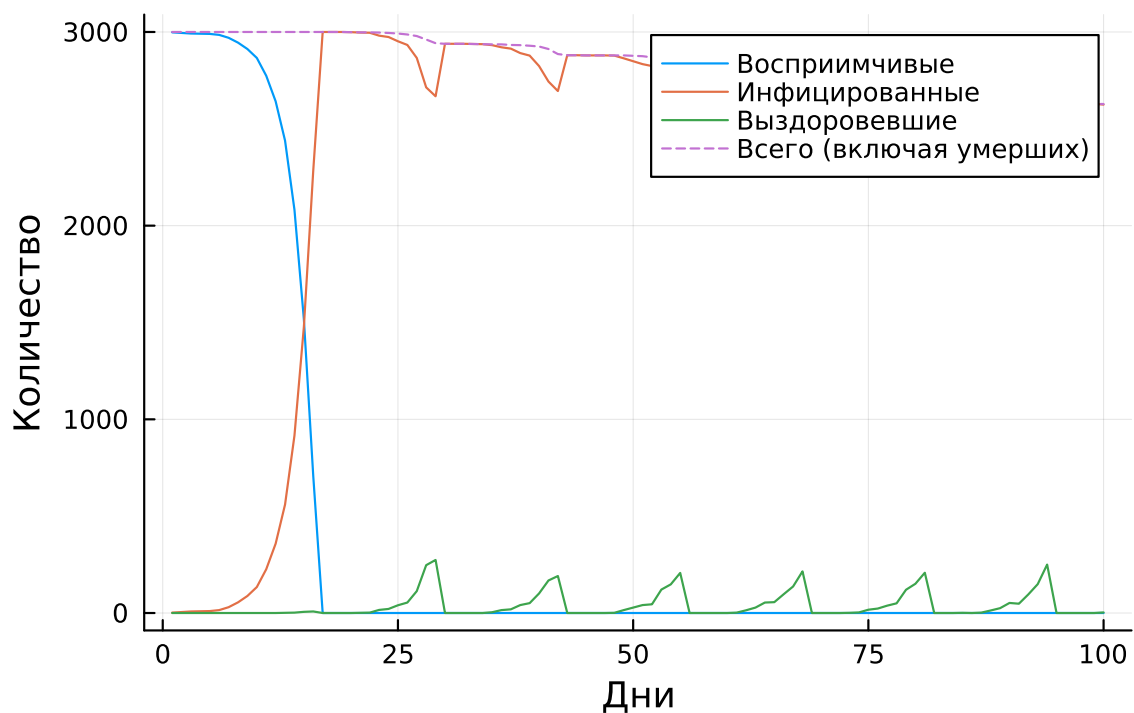
Сохраняем данные в формате JLD2.

```
@save datadir("sir_basic_agent.jld2") agent_df
@save datadir("sir_basic_model.jld2") model_df
```

5.5 Визуализация

Строим график динамики эпидемии.

```
Plots.gr(fmt=:png)
plot(agent_df.time, agent_df.susceptible,
      label = "Восприимчивые",
      xlabel = "Дни", ylabel = "Количество")
plot!(agent_df.time, agent_df.infected, label = "Инфицированные")
plot!(agent_df.time, agent_df.recovered, label = "Выздоровевшие")
plot!(agent_df.time, model_df.total, label = "Всего (включая умерших)",
```



Сохраняем график.

```
savefig(plotsdir("sir_basic_dynamics.png"))
```

```
"C:\\Users\\bogda\\work\\study\\2026-1\\2026-1--study--simulationmodeling\\labs\\lab04\\project\\plots\\sir_basic_dynamics"
```

Создаю скрипт sir_scan_beta. При $\beta \geq 0.4$ все инфицируются (рис. 5.1).

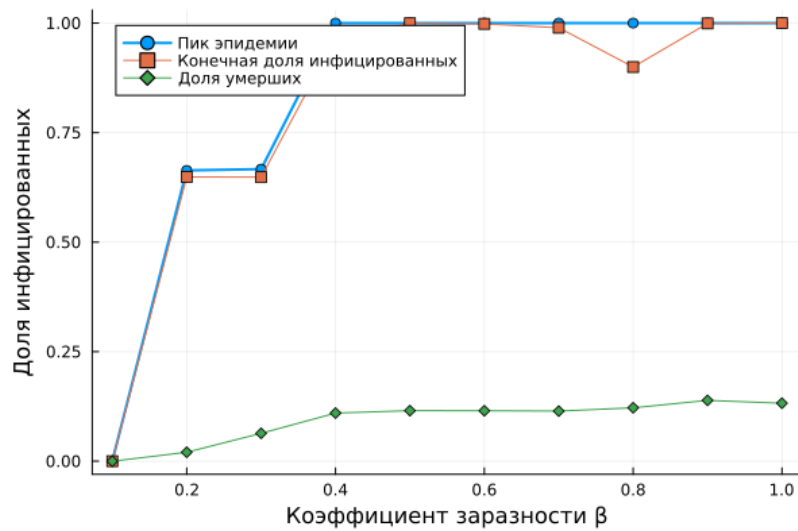


Рисунок 5.1: выполнение

6 Сканирование параметра β в модели SIR

Исследуем влияние коэффициента заразности на динамику эпидемии.

```
using DrWatson
@quickactivate "project"

using Agents, DataFrames, Plots, CSV, Random
```

```
└ Warning: DrWatson could not find find a project file by recursively c
⊠ (given dir: C:\Users\bogda)
└ @ DrWatson C:\Users\bogda\.julia\packages\DrWatson\2QF5p\src\proje
```

Подключаем файл с определением модели.

```
include(sourcedir("sir_model.jl"))
```

```
total_count (generic function with 1 method)
```

6.1 Функция для запуска одного эксперимента

Принимает параметры и возвращает ключевые метрики.

```

function run_experiment(p)
    beta = p[:beta]
    ⍺_und = fill(beta, 3)
    ⍺_det = fill(beta/10, 3)
    model = initialize_sir(;
        Ns = p[:Ns],
        ⍺_und = ⍺_und,
        ⍺_det = ⍺_det,
        infection_period = p[:infection_period],
        detection_time = p[:detection_time],
        death_rate = p[:death_rate],
        reinfection_probability = p[:reinfection_probability],
        Is = p[:Is],
        seed = p[:seed],
        n_steps = p[:n_steps],
    )
    infected_fraction(model) =
        count(a.status == :I for a in allagents(model)) / nagents(m

    peak_infected = 0.0
    for step = 1:p[:n_steps]
        agent_ids = collect(allids(model))
        for id in agent_ids
            agent = try
                model[id]
            catch
                nothing
            end
        end
    end

```

```

        if agent != nothing
            sir_agent_step!(agent, model)
        end
    end

    frac = infected_fraction(model)
    if frac > peak_infected
        peak_infected = frac
    end
end

    final_infected = infected_fraction(model)
    final_recovered = count(a.status == :R for a in allagents(model)
    total_deaths = sum(p[:Ns]) - nagents(model)

    return (
        peak = peak_infected,
        final_inf = final_infected,
        final_rec = final_recovered,
        deaths = total_deaths,
    )
end

```

run_experiment (generic function with 1 method)

6.2 Параметры эксперимента

Диапазон значений β и повторы для разных seed.

```
beta_range = 0.1:0.1:1.0
seeds = [42, 43, 44]
```

3-element Vector{Int64}:

42

43

44

6.3 Генерация списка параметров

Создаём все комбинации β и seed.

```
params_list = []
for b in beta_range
    for s in seeds
        push!(
            params_list,
            Dict(
                :beta => b,
                :Ns => [1000, 1000, 1000],
                :infection_period => 14,
                :detection_time => 7,
                :death_rate => 0.02,
                :reinfection_probability => 0.1,
                :Is => [0, 0, 1],
                :seed => s,
                :n_steps => 100,
            ),
        )
    end
end
```

```
end  
end
```

6.4 Запуск экспериментов

Выполняем все прогоны и собираем результаты.

```
results = []  
for params in params_list  
    data = run_experiment(params)  
    push!(results, merge(params, Dict{pairs(data)}))  
    println("XXXXXXXXXXXXXXXXX beta = $(params[:beta]), seed = $(pa  
end
```

```
XXXXXXXXXXXXXXXXX beta = 0.1, seed = 42  
XXXXXXXXXXXXXXXXX beta = 0.1, seed = 43  
XXXXXXXXXXXXXXXXX beta = 0.1, seed = 44  
XXXXXXXXXXXXXXXXX beta = 0.2, seed = 42  
XXXXXXXXXXXXXXXXX beta = 0.2, seed = 43  
XXXXXXXXXXXXXXXXX beta = 0.2, seed = 44  
XXXXXXXXXXXXXXXXX beta = 0.3, seed = 42  
XXXXXXXXXXXXXXXXX beta = 0.3, seed = 43  
XXXXXXXXXXXXXXXXX beta = 0.3, seed = 44  
XXXXXXXXXXXXXXXXX beta = 0.4, seed = 42  
XXXXXXXXXXXXXXXXX beta = 0.4, seed = 43  
XXXXXXXXXXXXXXXXX beta = 0.4, seed = 44  
XXXXXXXXXXXXXXXXX beta = 0.5, seed = 42  
XXXXXXXXXXXXXXXXX beta = 0.5, seed = 43  
XXXXXXXXXXXXXXXXX beta = 0.5, seed = 44
```

XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.6, seed = 42
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.6, seed = 43
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.6, seed = 44
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.7, seed = 42
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.7, seed = 43
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.7, seed = 44
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.8, seed = 42
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.8, seed = 43
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.8, seed = 44
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.9, seed = 42
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.9, seed = 43
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.9, seed = 44
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 1.0, seed = 42
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 1.0, seed = 43
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 1.0, seed = 44

6.5 Сохранение результатов

Сохраняем данные всех прогонов в CSV.

```
df = DataFrame(results)
CSV.write(datadir("beta_scan_all.csv"), df)
```

```
"C:\\Users\\bogda\\work\\study\\2026-1\\2026-1--study--
simulationmodeling\\labs\\lab04\\project\\data\\beta_scan_all.csv"
```

6.6 Усреднение по повторам

Усредняем метрики для каждого значения β .

```
using Statistics
grouped = combine(
    groupby(df, [:beta]),
    :peak => mean => :mean_peak,
    :final_inf => mean => :mean_final_inf,
    :deaths => mean => :mean_deaths,
)
```

	beta	mean_peak	mean_final_inf	mean_deaths
	Float64	Float64	Float64	Float64
1	0.1	0.000444444	0.0	0.0
2	0.2	0.663745	0.648685	61.3333
3	0.3	0.666666	0.648379	191.0
4	0.4	1.0	0.965633	329.333
5	0.5	1.0	0.999619	346.0
6	0.6	1.0	0.998094	345.333
7	0.7	1.0	0.989378	343.0
8	0.8	1.0	0.899806	365.667
9	0.9	1.0	0.999227	416.333
10	1.0	1.0	1.0	397.0

6.7 Визуализация результатов

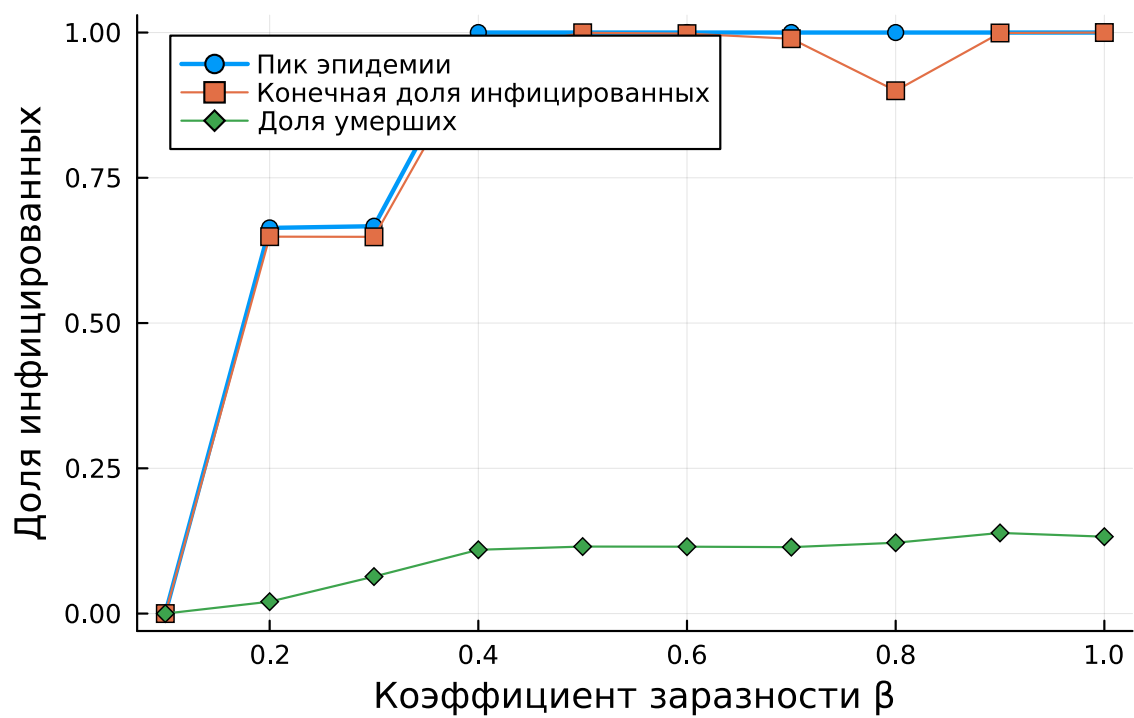
Строим график зависимости метрик от β .

```
plot(
    grouped.beta,
    grouped.mean_peak,
    label = "👤 🏠🏠🏠🏠🏠",
    xlabel = "🏠🏠🏠🏠🏠🏠 🏠🏠🏠🏠🏠 🏠",
)
```

```

        ylabel = "Среднее количество зараженных",
        marker = :circle,
        linewidth = 2,
    )
    plot!(
        grouped.beta,
        grouped.mean_final_inf,
        label = "Средняя конечная доля инфицированных",
        marker = :square,
    )
    plot!(grouped.beta, grouped.mean_deaths ./ 3000, label = "Средняя доля умерших")

```



Сохраняем график.

```

savefig(plotsdir("beta_scan.png"))

```

```
println("XXXXXXXXXX XXXXXXXXXXXX X data/beta_scan_all.csv X plots/beta_scan
```

```
XXXXXXXXXX XXXXXXXXXXXX X data/beta_scan_all.csv X plots/beta_scan.png
```

Создаём скрипт `sir_migration_effect`, на котором видно, что при интенсивности миграции 0.1 все инфицируются (рис. 6.1)

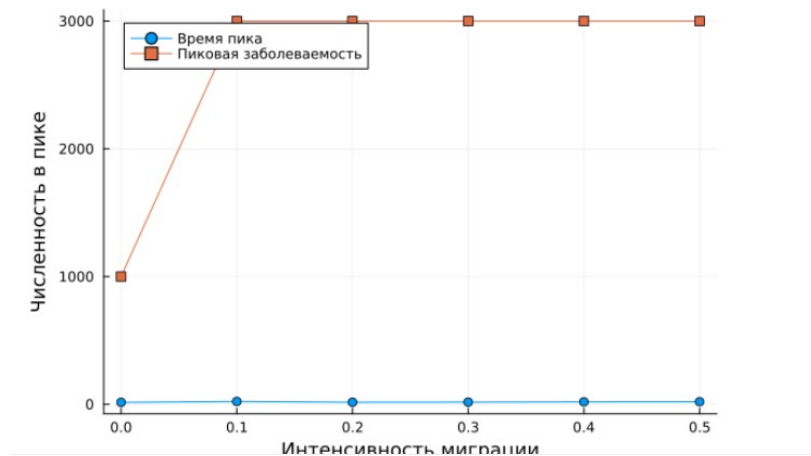


Рисунок 6.1: migration

```
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots, CSV, Random

include(sourcedir("sir_model.jl"))
```

```
└ Warning: DrWatson could not find find a project file by recursively cl
X (given dir: C:\Users\bogda)
└ @ DrWatson C:\Users\bogda\.julia\packages\DrWatson\2QF5p\src\proje
total_count (generic function with 1 method)
```

Функция для создания матрицы миграции с заданной интенсивностью

```

function create_migration_matrix(C, intensity)
    M = ones(C, C) .* intensity ./ (C-1)
    for i = 1:C
        M[i, i] = 1 - intensity
    end
    return M
end

```

create_migration_matrix (generic function with 1 method)

Функция для измерения времени достижения пика

```

function peak_time(p)
    migration_rates = create_migration_matrix(p[:C], p[:migration_i
    model = initialize_sir(;
        Ns = p[:Ns],
        ⌀_und = p[:⌀_und],
        ⌀_det = p[:⌀_det],
        infection_period = p[:infection_period],
        detection_time = p[:detection_time],
        death_rate = p[:death_rate],
        reinfection_probability = p[:reinfection_probability],
        Is = p[:Is],
        seed = p[:seed],
        migration_rates = migration_rates,
    )

    infected_frac(model) = count(a.status == :I for a in allagents(
    peak = 0.0
    peak_step = 0

```

```

for step = 1:p[:n_steps]
    agent_ids = collect(allids(model))
    for id in agent_ids
        agent = try
            model[id]
        catch
            nothing
        end
        if agent != nothing
            sir_agent_step!(agent, model)
        end
    end

    frac = infected_frac(model)
    if frac > peak
        peak = frac
        peak_step = step
    end
end

    return (peak_time = peak_step, peak_value = peak)
end

```

peak_time (generic function with 1 method)

Сканирование интенсивности миграции

```

migration_intensities = 0.0:0.1:0.5
seeds = [42, 43, 44]

params_list = []
for mig in migration_intensities
    for s in seeds
        push!(
            params_list,
            Dict(
                :migration_intensity => mig, # XXXXXX
                :C => 3,
                :Ns => [1000, 1000, 1000],
                :I_und => [0.5, 0.5, 0.5],
                :I_det => [0.05, 0.05, 0.05],
                :infection_period => 14,
                :detection_time => 7,
                :death_rate => 0.02,
                :reinfection_probability => 0.1,
                :Is => [1, 0, 0],
                :seed => s,
                :n_steps => 150,
            ),
        )
    end
end
end

```

Запуск

```

results = []
for params in params_list
    data = peak_time(params)
    push!(results, merge(params, Dict(pairs(data))))
    println(
        "XXXXXXXXXXXXXXXX migration_intensity = $(params[:migration_intensity])
    )
end

```

```

XXXXXXXXXXXXXXXX migration_intensity = 0.0, seed = 42
XXXXXXXXXXXXXXXX migration_intensity = 0.0, seed = 43
XXXXXXXXXXXXXXXX migration_intensity = 0.0, seed = 44
XXXXXXXXXXXXXXXX migration_intensity = 0.1, seed = 42
XXXXXXXXXXXXXXXX migration_intensity = 0.1, seed = 43
XXXXXXXXXXXXXXXX migration_intensity = 0.1, seed = 44
XXXXXXXXXXXXXXXX migration_intensity = 0.2, seed = 42
XXXXXXXXXXXXXXXX migration_intensity = 0.2, seed = 43
XXXXXXXXXXXXXXXX migration_intensity = 0.2, seed = 44
XXXXXXXXXXXXXXXX migration_intensity = 0.3, seed = 42
XXXXXXXXXXXXXXXX migration_intensity = 0.3, seed = 43
XXXXXXXXXXXXXXXX migration_intensity = 0.3, seed = 44
XXXXXXXXXXXXXXXX migration_intensity = 0.4, seed = 42
XXXXXXXXXXXXXXXX migration_intensity = 0.4, seed = 43
XXXXXXXXXXXXXXXX migration_intensity = 0.4, seed = 44
XXXXXXXXXXXXXXXX migration_intensity = 0.5, seed = 42
XXXXXXXXXXXXXXXX migration_intensity = 0.5, seed = 43
XXXXXXXXXXXXXXXX migration_intensity = 0.5, seed = 44

```

Сохраняем все прогоны

```
df = DataFrame(results)
CSV.write(datadir("migration_scan_all.csv"), df)
```

```
"C:\\Users\\bogda\\work\\study\\2026-1\\2026-1--study--
simulationmodeling\\labs\\lab04\\project\\data\\migration_scan_all.
```

Усреднение по повторам

```
using Statistics
grouped = combine(
    groupby(df, [:migration_intensity]),
    :peak_time => mean => :mean_peak_time,
    :peak_value => mean => :mean_peak_value,
)
```

	migration_intensity	mean_peak_time	mean_peak_value
	Float64	Float64	Float64
1	0.0	15.6667	0.333333
2	0.1	18.3333	1.0
3	0.2	20.3333	1.0
4	0.3	20.6667	1.0
5	0.4	17.3333	1.0
6	0.5	17.3333	1.0

Визуализация

```
plot(
    grouped.migration_intensity,
    grouped.mean_peak_time,
    marker = :circle,
    xlabel = "XXXXXXXXXXXXXXXX XXXXXXXX",
    ylabel = "XXXXXX XX XXXX (XXX)",
```

```

        label = "Время выявления",
    )
plot!(
    grouped.migration_intensity,
    grouped.mean_peak_value .* 3000,
    marker = :square,
    xlabel = "Время выявления",
    ylabel = "Интенсивность миграции",
    label = "Время выявления",
)
savefig(plotsdir("migration_effect.png"))

println("Время выявления \n data/migration_scan_all.csv \n plots/migration_effect.png")

```

```

Время выявления \n data/migration_scan_all.csv \n plots/migration_effect.png

```

Создаю скрипт `sir_optimize_parameters`. Получаю оптимальные параметры (рис. 6.2)

```

Best candidate found: [0.156542, 3.53029, 0.0921405]
Fitness: (0.00040, 0.00007) agg=0.00047
Оптимальные параметры:
β_und = 0.15654203610895967
Время выявления = 4 дней
Смертность = 0.09214047053238411
Достигнутые показатели:
Пик заболеваемости: 0.0004
Доля умерших: 6.666666666666667e-5

```

Рисунок 6.2: parameters

7 Оптимизация параметров модели SIR

Ищем оптимальные параметры для минимизации пиковой заболеваемости и смертности.

```
using DrWatson
@quickactivate "project"

using BlackBoxOptim, Random, Statistics
```

```
└ Warning: DrWatson could not find find a project file by recursively cl
⊠ (given dir: C:\Users\bogda)
└ @ DrWatson C:\Users\bogda\.julia\packages\DrWatson\2QF5p\src\projec
```

Подключаем файл с определением модели.

```
include(scrdir("sir_model.jl"))
```

```
total_count (generic function with 1 method)
```

7.1 Целевая функция

Минимизируем пиковую заболеваемость и долю умерших. $x[1]$: β_{und} (коэффициент заразности) $x[2]$: $detection_time$ (время выявления) $x[3]$: $death_rate$ (смертность)

```

function cost_multi(x)
    infected_frac(model) = count(a.status == :I for a in allagents(model))
    dead_count(model) = 3000 - nagents(model)
    replicates = 5
    peak_vals = Float64[]
    dead_vals = Int[]

    for rep = 1:replicates
        model = initialize_sir(;
            Ns = [1000, 1000, 1000],
            β_und = fill(x[1], 3),
            β_det = fill(x[1]/10, 3),
            infection_period = 14,
            detection_time = round(Int, x[2]),
            death_rate = x[3],
            reinfection_probability = 0.1,
            Is = [0, 0, 1],
            seed = 42 + rep,
            n_steps = 100,
        )

        peak_infected = 0.0

        for step = 1:100
            Agents.step!(model, 1)
            frac = infected_frac(model)
            if frac > peak_infected
                peak_infected = frac
            end
        end
    end
end

```

```

end

end

push!(peak_vals, peak_infected)
push!(dead_vals, dead_count(model))

end

return (mean(peak_vals), mean(dead_vals) / 3000) #  

end

```

7.2 Запуск оптимизации

```
result = bboptimize(
    cost_multi,
    Method = :borg_moea,
    FitnessScheme = ParetoFitnessScheme{2}(is_minimizing = true),
    SearchRange = [
        (0.1, 1.0),      # x_und
        (3.0, 14.0),     # detection_time
        (0.01, 0.1),     # death_rate
    ],
    NumDimensions = 3,
    MaxTime = 120,       # 2 000000
    TraceMode = :compact,
```

```
)
```

```
Starting optimization with optimizer BlackBoxOptim.BorgMOEA{EpsBoxDom
0.00 secs, 0 evals, 0 steps
pop.size=50 arch.size=0 n.restarts=0
```

```
Optimization stopped after 1 steps and 684.78 seconds
Termination reason: Max time (120.0 s) reached
Steps per second = 0.00
Function evals per second = 0.07
Improvements/step = NaN
Total function evaluations = 51
```

```
Best candidate found: [0.101363, 5.34156, 0.0672003]
```

```
Fitness: (0.00047, 0.00000) agg=0.00047
```

```
BlackBoxOptim.OptimizationResults("borg_moea", "Max time (120.0 s) re
1.0, :TargetFitness => nothing, :TraceMode => :compact, :FitnessScheme
50, :NumDimensions => :NotSpecified, :FitnessTolerance => 1.0e-
8, :TraceInterval => 0.5, :MaxStepsWithoutProgress => 10000, :MaxSteps
```

7.3 Результаты оптимизации

Извлекаем оптимальные параметры и достигнутые значения.

```
best = best_candidate(result)
fitness = best_fitness(result)
```

```
println("XXXXXXXXXX XXXXXXXX:")
println("λ_und = $(best[1])")
println("XXXXX XXXXXXXX = $(round{Int, best[2]}) XXXX")
println("XXXXXXXXXX = $(best[3])")
println()
println("XXXXXXXXXX XXXXXXXX:")
println("XX XXXXXXXXXXXXXXXX: $(fitness[1])")
println("XXXX XXXXXXX: $(fitness[2])")
```

XXXXXXXXXX XXXXXXXX:

λ_und = 0.10136280278899229

XXXXX XXXXXXXX = 5 XXXX

XXXXXXXXXX = 0.06720033267324195

XXXXXXXXXX XXXXXXXX:

XX XXXXXXXXXXXXXXXX: 0.00046666666666666666

XXXX XXXXXXX: 0.0

Сохраняем результат оптимизации.

```
save(datadir("optimization_result.jld2"), Dict("best" => best, "fit"
```

И графики (рис. 7.1)

(пик % популяции). График (рис. 7.3)

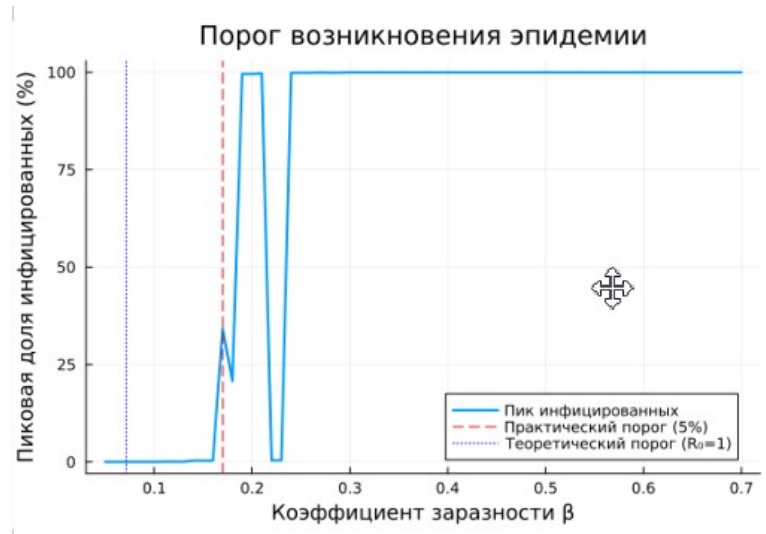


Рисунок 7.3: график

Занчение (рис. 7.4)

```
$ julia 02_threshold_analysis.jl
Activating project at `C:\Users\bogda\work\study\2026-1\
Минимальное  $\beta$  для эпидемии (пик > 5%): 0.17
Теоретический порог ( $R_0 = 1$ ):  $\beta = 0.071$ 
Результаты сохранены в plots/sir_threshold_analysis.png
```

Рисунок 7.4

Задайте разные значения beta для разных городов(рис. 7.5)

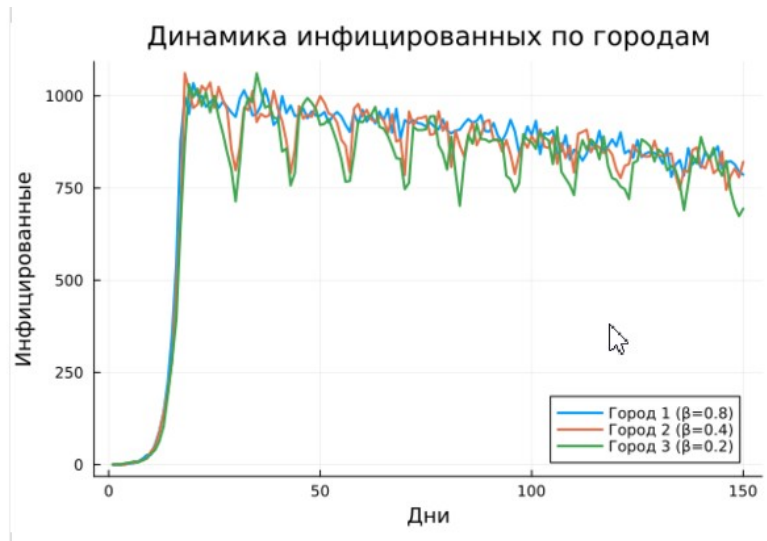


Рисунок 7.5

Исследую влияние интенсивности миграции на скорость распространения инфекции из одного города в другие. (рис. 7.6)

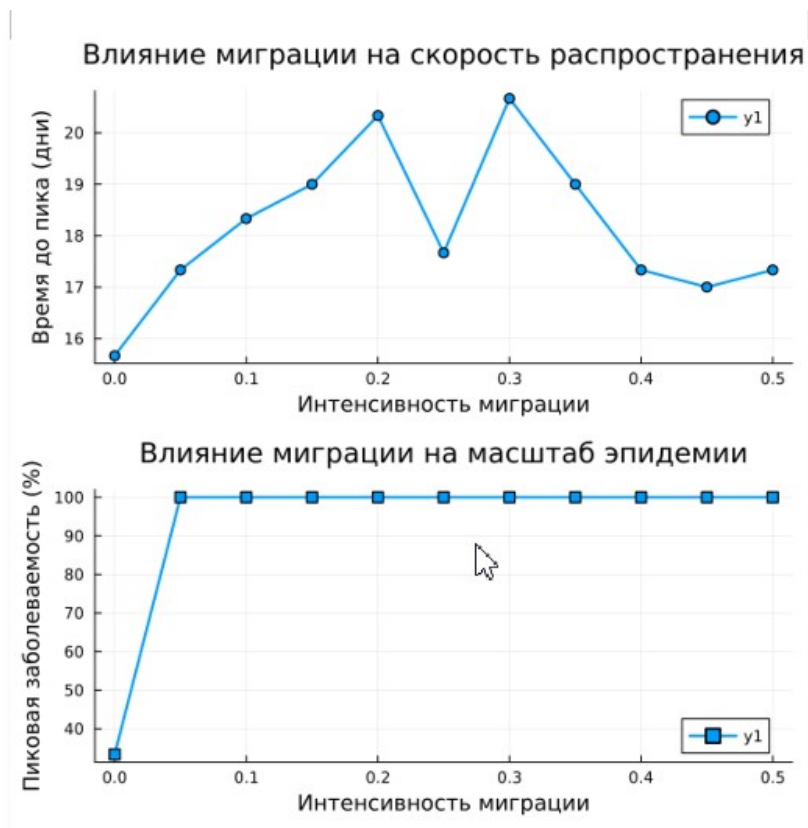


Рисунок 7.6

Минимальное время до пика (рис. 7.7)

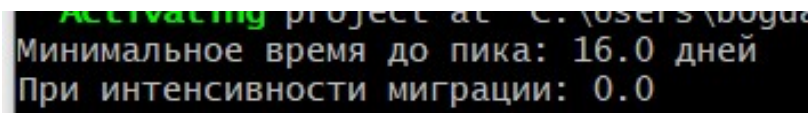


Рисунок 7.7

Модифицируйте модель, добавив возможность закрытия города (обнуление миграции из него) при превышении порога заболеваемости. Эффективность получается не высокой. (рис. 7.8)

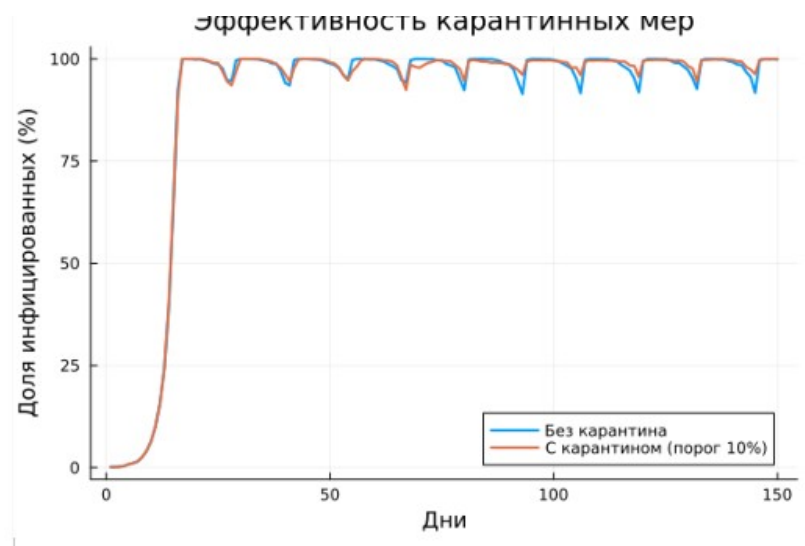


Рисунок 7.8

Считаем оптимальные параметры (рис. 7.9)

```
Activating project at `C:\Users\bogda\wor  
Город 1 закрыт на карантин на шаге 11
```

Рисунок 7.9: вывод кода

8 Выводы

Я познакомился с агентной моделью DaisyWorld, научился моделировать сложную систему, которая меняется под действием сущностей (агентов)

Список литературы